✦ AI BRAND VOICE GENERATOR ✦

PROJECT DOCUMENTATION

Laptop development setup: VS Code • Flask • LangChain • Google Gemini

✦ 1. Project Overview

AI Brand Voice Generator is a full-stack generative-AI web application that creates a professional brand voice guide from a brand description, selected tone and target audience. The frontend uses HTML, CSS and JavaScript; Flask provides the backend API; LangChain organizes the AI workflow; and Google Gemini generates the brand voice.

✦ 2. Project Objective

- Generate brand personality.
- Create communication-style guidelines.
- Suggest words to use and words to avoid.
- Generate example brand messages.
- Provide a simple, modern and responsive user experience.

✦ 3. Development Environment

Code editor: Visual Studio Code (VS Code)
Language: Python
Backend: Flask
AI framework: LangChain
AI model: Google Gemini
Frontend: HTML, CSS, JavaScript
Version control: Git / GitHub
Deployment: Render
Package manager: pip

✦ 4. Main Features

- Brand description input with a 500-character limit.
- Brand tone selector.
- Target audience selector.
- AI-generated personality.
- AI-generated communication style.
- Words to use and words to avoid.
- Example messages.
- Loading state during generation.
- Copy generated result.
- Error handling for invalid input and AI/API failures.

✦ 5. Brand Tone Options

- Friendly
- Professional
- Bold
- Playful
- Luxury
- Minimal

✦ 6. Target Audience Options

- General audience
- Young adults
- Professionals
- Entrepreneurs
- Creators
- Businesses

✦ 7. Application Flow

User → HTML/CSS/JavaScript → Flask /generate → LangChain → Google Gemini → JSON result → Website

✦ 8. Project Structure

```
ai-brand-voice-generator/
├── app.py
├── requirements.txt
├── templates/
│   └── index.html
├── static/
│   ├── style.css
│   └── script.js
└── README.md
```

✦ 9. Backend – app.py

- Creates the Flask application and serves index.html.
- Receives description, tone and audience through POST /generate.
- Validates the submitted brand description.
- Uses LangChain to organize the prompt/model workflow.
- Uses Gemini to generate the brand voice.
- Returns the result as JSON to the frontend.
- Reads the API key from an environment variable rather than hard-coding it.

✦ 10. Frontend – HTML, CSS and JavaScript

- index.html contains the form and result layout.
- style.css creates the dark interface and purple visual style.
- script.js sends form data to /generate using fetch().
- The result is displayed without a page refresh.
- The Copy button copies the generated guide to the clipboard.

✦ 11. What LangChain Does

- LangChain is the AI application framework between Flask and Gemini.
- It helps organize prompts and model calls into a reusable workflow.
- It separates AI-generation logic from the Flask route.
- It makes future additions such as structured output, prompt templates or multi-step AI workflows easier.

Simple explanation: Flask handles the web/API, LangChain organizes the AI workflow, and Gemini generates the content.

✦ 12. API Key Security

- Keep the Gemini API key in an environment variable.
- Do not place the real key inside app.py.
- Do not put the key in frontend JavaScript.
- Do not commit the key to GitHub.
- Add the key through Render Environment Variables for deployment.

✦ 13. requirements.txt

Typical final dependencies:

```
Flask
gunicorn
langchain
langchain-google-genai
python-dotenv
```

✦ 14. Run Locally in VS Code

1. Open the project folder in VS Code.
2. Open the VS Code integrated terminal.
3. Create/activate a Python virtual environment if required.
4. Install dependencies with: pip install -r requirements.txt
5. Set the Gemini API key as an environment variable.
6. Start the Flask application using the project's configured command.
7. Open the local Flask address in a browser.
8. Enter a brand description, choose tone and audience, then click Generate Brand Voice.

✦ 15. GitHub and Render Deployment

9. Save all changes in VS Code.
10. Push the updated files to GitHub.
11. Render pulls the latest commit.
12. Render installs packages from requirements.txt.
13. Render starts the Flask application with Gunicorn.
14. Render supplies the Gemini API key through environment variables.
15. After deployment, test the live Render service.

✦ 16. Testing Checklist

- Website opens successfully.
- Description field accepts input.
- 500-character counter works.

- Tone selector shows all six options.
- Audience selector shows all six options.
- Generate button shows loading state.
- AI result displays all five sections.
- Copy button works.
- Empty description displays an error.
- AI/API errors display a useful message.
- Render service shows Live.

✦ 17. Sample Test

- Brand description: EcoGlow is a modern skincare brand creating natural, affordable skincare products using plant-based ingredients for young adults who want simple and healthy routines.
- Tone: Friendly
- Target audience: Young adults

✦ 18. Future Improvements

- Add more tone and audience presets.
- Allow regeneration of individual sections.
- Add downloadable brand guides.
- Add saved brand voices and user accounts.
- Add multilingual generation.
- Add structured-output validation.
- Add analytics for generated brand voices.

✦ 19. Conclusion

This project demonstrates practical generative-AI development with a modern frontend, Flask backend, LangChain workflow, Google Gemini integration, GitHub version control and Render deployment. It is suitable as a portfolio project because it shows both web development and real AI integration.